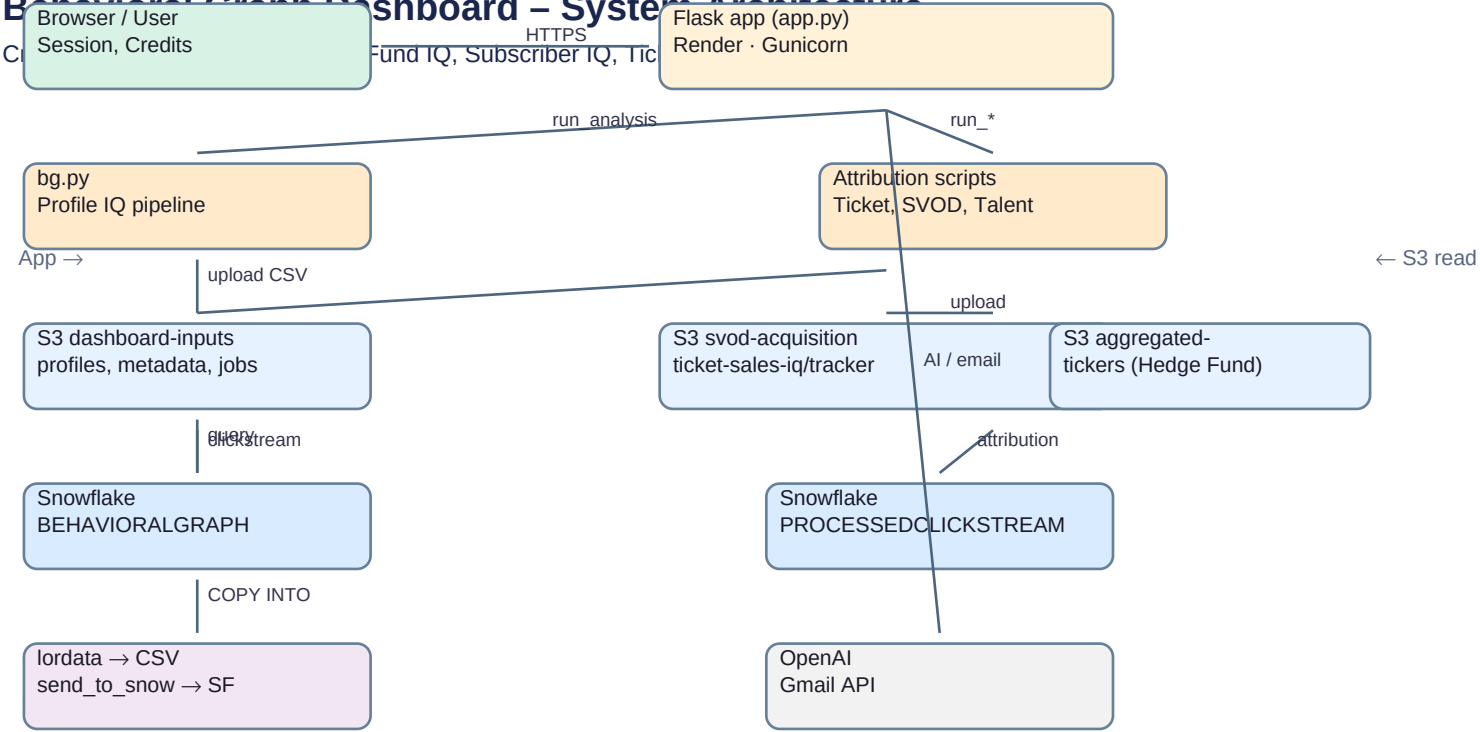


# Behavioral Graph Dashboard – System Architecture



## 1. High-level data flow

- User opens dashboard in browser, logs in (session; users.json or S3). Credits tracked per user.
- Profile IQ: User submits job (brand, date ranges, filters) → app runs `run_analysis()` in background → `bg.py` connects to Snowflake BEHAVIORALGRAPH.
- Hedge Fund IQ: Ticker list and CSV data from S3 aggregated-tickers. Ticker images, profile mappings, SEC actuals from S3 dashboard-inputs.
- Subscriber IQ: Profile list and CSV data from S3 svod-acquisition. Attribution: user runs SVOD job → app runs `SVOD_Churn_Attribution.py` →
- Ticket Sales IQ: List and data from S3 ticket-sales-iq. Ticket Sales Tracker: S3 ticket-sales-tracker. Attribution jobs run `Ticket_Sales_Attribution.py`.
- Netflix Ranker: App queries Snowflake BEHAVIORALGRAPH.PUBLIC.NETFLIX (by date, show, season, episode). Results cached in S3 dashboard-inputs.

## 2. Components (in-depth)

### Render (host)

- Flask app (`app.py`): All HTTP routes, `/api/*` endpoints, session-based auth, user/credit management, job queue (in-memory jobs dict + S3 jobs).
- `bg.py`: Behavioral graph pipeline. Connects to Snowflake (BEHAVIORALGRAPH, warehouse BEHAVIORGRAPH6X), performs universe search.
- Attribution scripts: `Talent_Theater_Attribution.py`, `Ticket_Sales_Attribution.py`, `SVOD_Churn_Attribution.py`, etc. Each connects to Snowflake.

### AWS S3 (region us-east-2)

- dashboard-inputs: Profile result CSVs (per job); metadata/ (`ticker_images_cache.json`, `ticker_profile_mappings.json`, `hedge_fund_sec_actuals.json`).
- svod-acquisition: Subscriber IQ CSV files.
- ticket-sales-iq: Ticket Sales IQ CSV files (talent-to-theater attribution).
- ticket-sales-tracker: Ticket Sales Tracker CSV files (movie viewers → theater).
- aggregated-tickers: Hedge Fund IQ ticker CSV files.

### Snowflake

- BEHAVIORALGRAPH database, BEHAVIORGRAPH6X warehouse: Raw clickstream tables (loaded by `send_to_snow.py` from lordata CSV).
- PROCESSEDCLICKSTREAM database: Attribution warehouses (e.g. TICKETS\_SALES\_WH\_6XL, ATTRIBUTIONPROCESSING) for Ticket Sales IQ.

### Data ingestion (offline)

- lordata\_part1 / lordata\_part2: Raw clickstream data → normalized CSV (columns e.g. `visit_ts`, `browser`, `platform`, `url`, `uid`, `Delivered`).
- `send_to_snow.py`: Scans local CSV folder, PUT to Snowflake stage, COPY INTO clickstream tables (CLICKBRAND\_2XL warehouse).

### External

- OpenAI: AI-generated insights, persona, marketing strategy, business deck, profile comparison (API key in env).
- Gmail API: Welcome and password-reset emails (OAuth tokens stored by app).

### 3. Profile IQ job flow (step-by-step)

1. User submits via `/api/submit` (POST) with `project_name`, `brands`, `sample_start/end`, `behavior_start/end`, `filters`, `options`.
2. App creates `job_id` (UUID), stores job in memory (`jobs[job_id]`) and persists status to S3 `system/jobs_status.json`.
3. `run_analysis()` runs in background thread: imports `bg`, expands brands via `generate_brand_variations`, sets deterministic seed, optionally downloads data.
4. `bg.connect_snowflake()` (token or password auth). `perform_full_universe_scan()` for sample sizing. `bg.run_full_pipeline()` runs full behavioral graph.
5. App uploads result CSV to S3 `dashboard-inputs`, updates job status to `completed`, triggers `smart_cache_update` so profile list includes new file.
6. User sees job complete; can open profile (data loaded via presigned S3 URL or `/api/job-data`). Frontend renders demographics, behavioral charts.

### 4. IQ products (in-depth)

#### Profile IQ

- Run new analysis (brand, dates, filters) or open existing profile from S3 list. Demographics, behavioral categories, frequency (optional), Genomics.

#### Hedge Fund IQ

- Ticker-centric. Ticker list and CSV data from S3 `aggregated-tickers`. Metadata (images, profile mappings, SEC actuals) from `dashboard-inputs`.

#### Subscriber IQ

- Profile list from S3 `svod-acquisition` (merged with `dashboard-inputs` cache for single list). View CSV data (subscriber metrics). Run SVOD analysis.

#### Ticket Sales IQ

- List and data from S3 `ticket-sales-iq`. Talent-to-theater attribution. Run attribution job → `Ticket_Sales_Attribution.py` → Snowflake → result table.

#### Ticket Sales Tracker

- List and data from S3 `ticket-sales-tracker`. Movie viewers → theater attribution. Same pattern: run job → attribution script → Snowflake → S3.

#### Netflix Ranker

- Data from Snowflake `BEHAVIORALGRAPH.PUBLIC.NETFLIX`. API returns `by_date` (series), `by_date_season` (show+season), `by_date_episode`.

## 5. API endpoints (summary)

Auth: /login, /logout, /terms, /privacy, /health, /healthz, /ready (no auth).

Profile IQ: /api/submit (POST), /api/status/<job\_id>, /api/jobs (list; ?refresh=1 to sync S3), /api/job-data/<job\_id>, /api/download/<job\_id>, /api/pr

Hedge Fund IQ: /api/hedge-fund-iq/tickers (list), /api/hedge-fund-iq/data/<ticker>, /api/hedge-fund-iq/profile-mapping/<ticker>, /api/hedge-fund-iq

Subscriber IQ: List/data via /api/jobs (svod-acquisition prefix). /api/attribution/svod-acquisition (POST) to run attribution.

Ticket Sales IQ: /api/ticket-sales-iq/list, /api/ticket-sales-iq/data/<s3\_key>. Ticket Sales Tracker: /api/ticket-sales-tracker/list, /api/ticket-sales-trac

Netflix Ranker: /api/netflix-ranker (GET; optional force\_refresh).

Admin: /admin, /api/admin/users, /api/admin/refresh-cache, /api/admin/ticker-image, /api/admin/ticket-sales-metadata, purgatory release, Gmail

## 6. Auth, credits & admin

- Session-based auth; users and hashed passwords in users.json (or S3). Roles: admin, user (optional purgatory\_access).
- Credits: each user has a credit balance; running a Profile IQ job consumes credits. Admin can set balance, restore defaults.
- Admin: Create/edit/delete users, reset password, send welcome/reset emails (Gmail). Purgatory: uploaded files land in S3 purgatory/ until adm

## 7. Deployment & tech stack

- Host: Render (render.com). Web service from render.yaml / render-native.yaml; buildCommand (build.sh), startCommand (start.sh), healthChe
- Python: Flask, flask-cors, flask-socketio, pandas, numpy, snowflake-connector-python, boto3, openai, google-api-python-client, reportlab, etc. (
- Frontend: Single-page app in templates/index.html (Chart.js, Socket.IO for real-time). Admin UI in templates/admin.html.
- Config: Environment variables (APP\_USERNAME, APP\_PASSWORD, SNOWFLAKE\_\*, AWS\_ACCESS\_KEY\_ID, AWS\_SECRET\_ACCESS